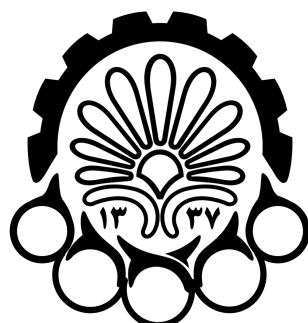




دانشگاه صنعتی امیرکبیر
(پلی تکنیک تهران)

دانشکده مهندسی کامپیوتر

پایان نامه پروژه کارشناسی

عنوان پروژه:

ترکیب تحلیل ساختاری کد با استدلال مدل های زبانی بزرگ و معماری چندعامله
برای کاهش هشدارهای کاذب در تحلیل آسیب پذیری کد

نگارنده:

علی ناصری نیا

استاد راهنما:

حمیدرضا شهریاری کاهکشی

تابستان ۱۴۰۵

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

اینجانب علی ناصری‌نیا متعهد می‌شوم که مطالب مندرج در این پایان‌نامه حاصل کار پژوهشی اینجانب تحت نظارت و راهنمایی اساتید دانشگاه صنعتی امیرکبیر بوده و به دستاوردهای دیگران که در این پژوهش از آن‌ها استفاده شده است مطابق مقررات و روال متعارف ارجاع و در فهرست منابع و مآخذ ذکر گردیده است. این پایان‌نامه قبلاً برای احراز هیچ مدرک هم‌سطح یا بالاتر ارائه نگردیده است.

در صورت اثبات تخلف در هر زمان، مدرک تحصیلی صادر شده توسط دانشگاه از درجه اعتبار ساقط بوده و دانشگاه حق پیگیری قانونی خواهد داشت.

کلیه نتایج و حقوق حاصل از این پایان‌نامه متعلق به دانشگاه صنعتی امیرکبیر است. هرگونه استفاده از نتایج علمی و عملی، واگذاری اطلاعات به دیگران یا چاپ و تکثیر، نسخه‌برداری، ترجمه و اقتباس از این پایان‌نامه بدون موافقت کتبی دانشگاه صنعتی امیرکبیر ممنوع است.

نقل مطالب با ذکر مآخذ بلامانع است.

علی ناصری‌نیا

امضا



سپاس‌گزاری

بدینوسیله مراتب قدردانی و امتنان خود را خدمت جناب آقای دکتر شهریاری‌کاهکشی، استاد راهنمای گرانقدر، که راهنمایی‌های خود را در تمام مراحل بر من ارزانی داشتند و بدون کمک‌های ایشان انجام این پروژه امکان‌پذیر نبود، ابراز و از زحمات ایشان بسیار سپاس‌گزار بوده و آرزومند توفیق روزافزون برایشان می‌باشم.

تابستان ۱۴۰۵

تقدیم

تقدیم به پدر و مادر عزیزم، که با مهربانی، صبر و حمایت بی‌دریغ خود همواره پشتوانه و چراغ راه من بوده‌اند. محبت و انگیزه‌ای که از شما دریافت کرده‌ام، همواره مشوق من در مسیر تحصیل و تلاش علمی بوده و بدون همراهی و دلگرمی‌های شما، دستیابی به این مرحله امکان‌پذیر نبوده است.

تابستان ۱۴۰۵

چکیده

مدیریت نایمن حافظه در زبان برنامه‌نویسی C و نبود سازوکار مدیریت خودکار، همواره منشأ آسیب‌پذیری‌های بحرانی نظیر نشتی حافظه (Memory Leak)، استفاده از حافظه پس از آزادسازی (Use-After-Free) و آزادسازی دوباره (Double-Free) در نرم‌افزارهای سیستمی بوده است. ابزارهای تحلیل پویا با ایجاد سربار اجرا همراه بوده و تحلیل‌گرهای ایستای تقریبی نیز دچار هشدارهای نادرست متعدد هستند. در این پژوهش، یک ابزار تحلیل ایستا مبتنی بر سامانه نوع خطی (Linear Type System) برای زیرمجموعه‌ای تورینگ‌کامل از زبان C (با نام Linear-C) طراحی و پیاده‌سازی شده است. در این روش، با مدل‌سازی رفتار حافظه در قالب قوانین نوع‌دهی و انتقال مالکیت متغیرها، چک کردن قوانین روی گراف کنترل جریان (CFG) برنامه انجام می‌گیرد تا خطاهای حافظه در زمان کامپایل و بدون هیچ‌گونه سربار زمان اجرا شناسایی شوند. ارزیابی تجربی ابزار روی ۲۲۰ نمونه آزمایشی (شامل ۱۰۰ کد آسیب‌پذیر و ۱۲۰ کد ایمن) نشان می‌دهد که این روش با تحقق یادآوری ۱۰۰٪ (کشف تمامی خطاهای موجود و عدم وجود منفی نادرست) و دقت ۷۸/۱۲٪، راهکاری درست، قطعی و کارآمد برای ارتقای پایداری و ایمنی کدهای C فراهم می‌سازد.

واژه‌های کلیدی:

تحلیل ایستا، زبان برنامه‌نویسی C، سامانه نوع خطی، ایمنی حافظه، گراف کنترل جریان، انتزاع بدون هزینه

فهرست مطالب

فصل اول - مقدمه	۱
۱.۱ بیان مسئله	۲
۱.۲ اهمیت و ضرورت پژوهش	۴
۱.۳ اهداف پژوهش	۴
۱.۴ دستاوردهای پژوهش	۵
۱.۵ روش پیشنهادی	۶
۱.۶ ساختار پایان نامه	۶
فصل دوم - مبانی نظری و کارهای پیشین	۷
۲.۱ تحلیل ایستا و مفهوم هشدار نادرست	۸
۲.۲ ابزارهای تحلیل ایستای به کاررفته در این پژوهش	۸
۲.۳ تحلیل ایستا-پویا و رویکردهای ترکیبی	۹
۲.۴ مدل های زبانی بزرگ و عامل های نرم افزاری	۹
۲.۵ استدلال مبتنی بر شواهد	۱۰
۲.۶ مجموعه داده جولیت	۱۰
۲.۷ کارهای پیشین	۱۱
۲.۸ جایگاه کار پیشنهادی	۱۲
فصل سوم - طراحی و پیاده سازی سامانه	۱۳
۳.۱ معماری کلی	۱۴
۳.۲ فاز یک: تحلیل ایستا	۱۴
۳.۳ فاز یک: همبسته سازی یافته ها	۱۵
۳.۴ فاز دو: داوری چندعامله	۱۵
۳.۵ بسته شواهد	۱۶
۳.۶ پاک سازی کد و جلوگیری از نشت برچسب	۱۷
۳.۷ سیاست تطبیق یافته ها و معیارها	۱۸
۳.۹ شمای گزارش و نمونه های واقعی	۱۸
۳.۱۰ زیرساخت اجرا و تکرارپذیری	۲۰

فصل چهارم - ارزیابی تجربی ۲۱

- ۴.۱ پرسش‌های پژوهش ۲۲
- ۴.۲ مجموعه داده و آماده‌سازی آن ۲۲
- ۴.۳ طراحی آزمایش‌ها ۲۳
- ۴.۴ معیارها و سیاست تطبیق ۲۳
- ۴.۵ نتایج اصلی ۲۳
- ۴.۶ تحلیل کیف تصمیم ۲۵
- ۴.۷ نتایج به تفکیک نوع آسیب‌پذیری ۲۶
- ۴.۸ بررسی در مقیاس کامل ۲۷
- ۴.۹ ارزیابی موتور شواهد در پایتون ۲۷
- ۴.۱۰ ارزیابی موتور شواهد در جولیت و دیویگن ۲۸
- ۴.۱۱ تهدیدهای روایی ۲۹

فصل پنجم - نتیجه‌گیری و کارهای آینده ۳۰

- ۵.۱ جمع‌بندی ۳۱
- ۵.۲ محدودیت‌ها ۳۱
- ۵.۳ کارهای آینده ۳۱

فهرست مراجع ۳۳

فهرست شکل‌ها

- شکل ۱.۱ - نمونه‌ای از کد آسیب‌پذیر از نوع سرریز عدد صحیح ۳
- شکل ۲.۱ - نسخه اصلاح‌شده همان سناریو با بررسی پیش از محاسبه ۳
- شکل ۳.۱ - نمای کلی سامانه پیشنهادی و جایگاه داوری چندعامله مبتنی بر شواهد ۴
- شکل ۱.۲ - نسخه آسیب‌پذیر یک سناریوی جولیت ۱۰
- شکل ۲.۲ - نسخه سالم همان سناریو؛ تنها تفاوت، یک شرط کنترلی است ۱۱
- شکل ۱.۳ - معماری سه‌فاز سامانه و جریان داده میان فازها ۱۴
- شکل ۲.۳ - نمونه ساده‌شده بسته شواهد جستجوگر ۱۶
- شکل ۳.۳ - قطعه کد پس از پاک‌سازی؛ حاشیه‌نویسی‌ها محو و شناسه‌های افشاگر بازنام‌گذاری شده‌اند ۱۷
- شکل ۴.۳ - زنجیره ادغام یافته‌های تکراری و تشکیل گروه‌های بازبینی ۱۵
- شکل ۱.۴ - مقایسه دقت، یادآوری و نرخ هشدار روی فایل سالم در چهار پیکربندی ۲۴
- شکل ۲.۴ - ماتریس سردرگمی چهار پیکربندی در دانه‌بندی فایل و یافته ۲۴
- شکل ۳.۴ - اثر شواهد ساختاری بر تصمیم‌های عامل زبانی ۲۵

فهرست جدول‌ها

- جدول ۲.۱ - ویژگی ابزارهای تحلیل ایستای به‌کاررفته در فاز یک ۸
- جدول ۲.۲ - جایگاه کار پیشنهادی در میان کارهای پیشین ۱۲
- جدول ۳.۱ - سه خروجی ممکن عامل ارزیاب ۱۵
- جدول ۳.۲ - سیاست تطبیق یافته‌ها با برجسب واقعی ۱۸
- جدول ۳.۳ - شمای گزارش نهایی سامانه ۱۹
- جدول ۳.۴ - دامنه پیشنهاد و نمونه‌های واقعی به‌کاررفته در گزارش ۱۹
- جدول ۴.۱ - مشخصات مجموعه داده ارزیابی و پیکربندی آزمایش ۲۲
- جدول ۴.۲ - نتایج اصلی چهار آزمایش روی ۶۰۰ نمونه متعادل ۲۳
- جدول ۴.۳ - قیف تصمیم عامل زبانی در هر پیکربندی ۲۵
- جدول ۴.۴ - نتایج به تفکیک نوع آسیب‌پذیری ۲۶
- جدول ۴.۵ - نتایج موتور قطعی روی محک ۴۸ نمونه‌ای پایتون ۲۷
- جدول ۴.۶ - نتایج موتور قطعی روی جولیت و دیویگن ۲۸

فصل اول

مقدمه

۱.۱ بیان مسئله

تحلیل ایستا^۱ یکی از پرکاربردترین رویکردها برای یافتن آسیب‌پذیری‌های امنیتی در نرم‌افزار است. در این رویکرد، کد پروژه بدون اجرا شدن و با استفاده از تحلیل نحوی، کنترل جریان داده و مجموعه‌ای از قواعد از پیش تعریف‌شده بررسی می‌شود و ابزار، محل‌هایی از کد را که ممکن است دربردارنده خطا باشند به‌عنوان هشدار گزارش می‌کند. مزیت اصلی این رویکرد آن است که نیازی به اجرای برنامه یا آماده‌سازی ورودی‌های خاص نبوده و در نتیجه می‌توان آن را در مراحل ابتدایی چرخه توسعه و بر روی کل مخزن نرم‌افزاری به کار برد. به همین دلیل، ابزارهایی مانند [۱] cppcheck و [۲] flawfinder در فرآیندهای یکپارچه ساخت و زنجیره‌های ابزار توسعه جایگاه تثبیت‌شده‌ای پیدا کرده‌اند.

با این حال، مهم‌ترین چالش عملی ابزارهای تحلیل ایستا، نرخ بالای هشدارهای نادرست^۲ است. هشدار نادرست یعنی ابزار محلی را به‌عنوان محل آسیب‌پذیری معرفی کرده است، در حالی که آن مسیر در واقع اجرا نمی‌شود، یا شرط لازم برای بهره‌برداری مهاجم هرگز برقرار نمی‌شود. بررسی‌های انجام‌شده روی پروژه‌های واقعی نشان می‌دهد که سهم قابل‌توجهی از هشدارهای این ابزارها در عمل نادرست است و همین موضوع سبب می‌شود توسعه‌دهندگان به‌تدریج به هشدارها بی‌اعتماد شوند و در نهایت ابزار را کنار بگذارند. بنابراین مسئله اصلی، افزایش تعداد هشدارها نیست، بلکه افزایش نسبت هشدارهای مفید و کاهش هشدارهای نادرست در عین حفظ توانایی کشف آسیب‌پذیری‌های واقعی است. در سال‌های اخیر، مدل‌های زبانی بزرگ^۳ توانسته‌اند درک معنایی از کد به دست آورند و این امکان را ایجاد کرده‌اند که بخشی از قضاوت‌های نیازمند فهم زمینه اجرای برنامه، به‌جای قواعد صرفاً نحوی، به این مدل‌ها واگذار شود. برای نمونه، بازبینی هشدار یک ابزار ایستا توسط یک مدل زبانی می‌تواند بررسی کند که آیا مسیر داده واقعاً قابل دسترس است، آیا مقدار مورد استفاده پیش از رسیدن به نقطه خطر اعتبارسنجی می‌شود و آیا نام توابع فراخوانی‌شده با الگوهای شناخته‌شده خطرناک همخوانی دارد یا خیر. با این حال، اتکال صرف به مدل زبانی نیز ریسک‌هایی به همراه دارد. این مدل‌ها ممکن است یافته‌ای را بدون شواهد کافی تأیید کنند، دلایل نادرست تولید کنند و در صورت مواجهه با کد مبهم، به‌جای اعلام عدم قطعیت، پاسخ قطعی تولید کنند [۱۰] [۱۵]. به همین دلیل، استفاده از آن‌ها به‌عنوان جایگزین کامل ابزارهای تحلیل ایستا، دست‌کم در ارزیابی‌های کمی، نتیجه رضایت‌بخشی به همراه نداشته است.

مسئله‌ای که این پژوهش بر آن تمرکز دارد، طراحی و ارزیابی یک سامانه ترکیبی است که در آن تحلیل ایستا نقش پیشنهاددهنده نامزدها را بر عهده می‌گیرد و دو عامل مستقل جستجوگر^۴ و ارزیاب^۵ نقش داور مبتنی بر شواهد را ایفا می‌کنند. هدف این سامانه آن است که هشدارهای نادرست ابزارهای ایستا را بدون از دست دادن آسیب‌پذیری‌های واقعی کاهش دهد. برای آنکه ادعای کاهش هشدار نادرست قابل دفاع باشد، لازم است ابتدا مشخص شود که معیار «کاهش»

^۱ Static Analysis

^۲ False Positive

^۳ Large Language Model

^۴ Scanner Agent

^۵ Verifier Agent

دقیقاً بر چه پایه‌ای سنجیده می‌شود. این موضوع در فصل چهارم با معرفی سیاست تطبیق یافته‌ها و تفکیک دانه‌بندی معیارها بررسی می‌شود، اما به‌طور خلاصه، در این پایان‌نامه دو دانه‌بندی متمایز گزارش می‌شود: دانه‌بندی یافته که تطابق میان هشدار ابزار و محل واقعی خطا را می‌سنجد، و دانه‌بندی فایل که میزان آلودگی هر فایل سالم به هشدار را اندازه می‌گیرد. تنها تعریف دقیق این دو دانه‌بندی است که اجازه می‌دهد درباره کاهش هشدار نادرست سخن گفته شود، زیرا در غیر این صورت، کاهش تعداد هشدارها می‌تواند صرفاً نتیجه حذف تصادفی یافته‌ها باشد.

CWE190_Integer_Overflow__int_rand_add_73b_bad.cpp

```
void badSink(list<int> dataList)
{
    int data = dataList.back();
    {
        /* POTENTIAL FLAW: Adding 1 to data could
           cause an overflow */
        int result = data + 1;
        printIntLine(result);
    }
}
```

شکل ۱.۱: نمونه‌ای از کد آسیب‌پذیر از نوع سرریز عدد صحیح

CWE190_Integer_Overflow__int_rand_add_73b_good.cpp

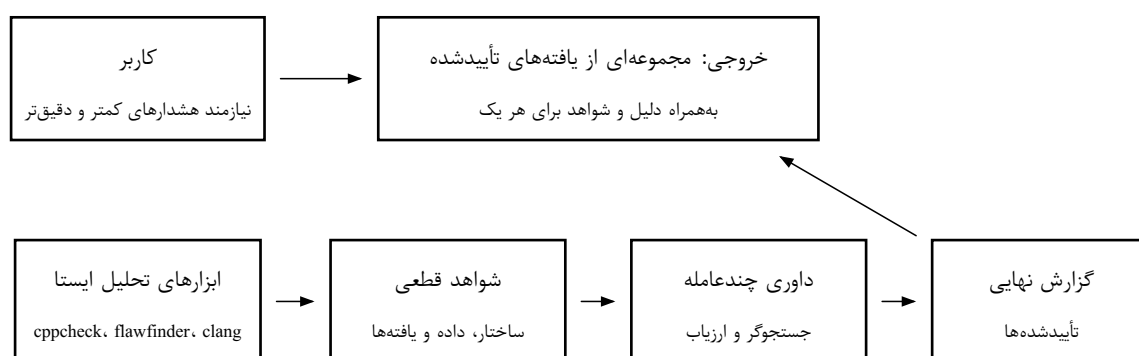
```
void goodB2GSink(list<int> dataList)
{
    int data = dataList.back();
    if (data < INT_MAX) {
        int result = data + 1;
        printIntLine(result);
    } else {
        printLine("data value is too large ...");
    }
}
```

شکل ۱.۲: نسخه اصلاح‌شده همان سناریو با بررسی پیش از محاسبه

۱.۲ اهمیت و ضرورت پژوهش

آسیب‌پذیری‌های ایمنی حافظه، از جمله سرریز بافر مبتنی بر هیپ^۶ و سرریز عدد صحیح^۷، همچنان از پرتکرارترین و خطرناک‌ترین دسته‌های آسیب‌پذیری در نرم‌افزارهای نوشته‌شده به زبان‌های C و C++ هستند. سازوکار این آسیب‌پذیری‌ها به‌گونه‌ای است که بسیاری از آن‌ها تنها با دنبال کردن جریان داده از منبع تا نقطه خطر و بررسی شرط‌های کنترل‌کننده آشکار می‌شوند؛ یعنی همان تحلیلی که ابزارهای ایستا انجام می‌دهند و به همین دلیل نیز بیشترین حجم هشدارهای نادرست از همین دسته تولید می‌شود. این تنش، یعنی ضرورت پوشش آسیب‌پذیری در کنار پرهیز از هشدار نادرست، مسئله مرکزی این پژوهش است.

اهمیت موضوع زمانی بیشتر می‌شود که هزینه بررسی دستی یک هشدار نادرست در نظر گرفته شود. هر هشدار نادرست، زمان توسعه‌دهنده را صرف خواندن کدی می‌کند که در نهایت سالم ارزیابی می‌شود و همین هزینه، انگیزه استفاده از ابزار را از بین می‌برد. در مقابل، یک آسیب‌پذیری که دیده نشود، ممکن است ماه‌ها یا سال‌ها در سامانه باقی بماند. بنابراین سامانه‌ای که بتواند با هزینه محاسباتی اندک، سیگنال اولیه ابزارهای ایستا را پالایش کند و تنها موارد دارای شاهد کافی را به توسعه‌دهنده نشان دهد، ارزش عملی روشنی دارد.



شکل ۱.۳: نمای کلی سامانه پیشنهادی و جایگاه داوری چندعامله مبتنی بر شواهد

۱.۳ اهداف پژوهش

هدف کلی این پژوهش، طراحی، پیاده‌سازی و ارزیابی کمی یک سامانه ترکیبی برای کاهش هشدارهای نادرست تحلیل ایستا در کد C و Python به کمک بازیابی مبتنی بر شواهد با دو عامل زبانی مستقل است. پشتیبانی نحوی از C++ برای پوشش کامل مجموعه جولیت حفظ شده است، اما دامنه زبانی اعلام‌شده پژوهش همان دو زبان پیشنهاد شده است. اهداف جزئی به شرح زیر بیان می‌شوند:

^۶Heap-based Buffer Overflow

^۷Integer Overflow

- طراحی معماری سه‌فاز شامل فاز شواهد قطعی، فاز داوری چندعامله جستجوگر و ارزیاب، و فاز ارزیابی مستقل، به‌گونه‌ای که مرز مسئولیت هر فاز روشن و قابل آزمون باشد.
 - تعریف بسته شواهد ساخت‌یافته شامل قطعه کد پاک‌سازی‌شده، پیام ابزارها و شواهد ساختاری، به‌منظور آنکه تصمیم‌عامل‌های زبانی بر پایه شواهد و نه بر پایه حدس گرفته شود.
 - پیاده‌سازی سازوکاری برای جلوگیری از نشت برچسب^۸ در مجموعه داده ارزیابی، به‌گونه‌ای که نتیجه آزمایش‌ها بازتاب توانایی واقعی سامانه باشد و نه توانایی خواندن پاسخ از درون فایل آزمون.
 - تدوین چارچوب ارزیابی تکرارپذیر شامل سیاست تطبیق یافته‌ها، تفکیک دانه‌بندی معیارها و گزارش نتایج به تفکیک نوع آسیب‌پذیری.
 - بررسی تجربی اثر هر جزء سامانه از طریق طراحی چهار آزمایش مقایسه‌ای و تحلیل نتایج آن‌ها.
- دامنه آسیب‌پذیری این پژوهش همان دامنه پیشنهاد است: در زبان C سرریز بافر و سرریز عدد صحیح، و در زبان Python تزریق فرمان، تزریق کد، تزریق قالب سمت سرور، بازسازی ناامن داده و استفاده خطرناک از رابط‌های برنامه‌نویسی کاربردی بررسی می‌شود. پشتیبانی موتور شواهد از تزریق SQL و پیمایش مسیر، قابلیت کمکی تحلیل است و در ارزیابی اصلی ادعایی درباره آن‌ها مطرح نمی‌شود. فهرست دقیق شناسه‌های CWE و نمونه‌های واقعی متناظر، همراه با موارد اصلاح‌شده شناسه‌های نادرست پیشنهاد، در بخش ۳.۹ آمده است.

۱.۴ دستاوردهای پژوهش

دستاوردهای اصلی این پژوهش به شرح زیر است:

- پیاده‌سازی یک سامانه سه‌فاز به‌صورت یک بسته نرم‌افزاری قابل اجرا، شامل اجرای ابزارهای تحلیل ایستا، استخراج ساختار نحوی، ارزیابی مسیر منبع تا نقطه مصرف، پیشنهاد فرضیه توسط عامل جستجوگر و تأیید یا رد هر فرضیه توسط عامل ارزیاب.
- ارائه ماژول پاک‌سازی کد که با محوکردن حاشیه‌نویسی‌ها در جای خود و بازنام‌گذاری شناسه‌های افشاگر، برچسب واقعی را از دید مدل پنهان می‌کند و در عین حال شماره خط گزارش‌شده توسط ابزارها را تغییر نمی‌دهد.
- ارائه چارچوب ارزیابی تکرارپذیر شامل چهار آزمایش مقایسه‌ای، سازوکار تطبیق گران‌خوار، معیارهای دانه‌بندی یافته و دانه‌بندی فایل و گزارش خودکار جدول مقایسه.
- به‌کارگیری مجموعه داده جولیت به‌عنوان مجموعه ارزیابی و گزارش نتایج آن بر روی مجموعه‌ای متعادل و لایه‌بندی‌شده از نظر نوع آسیب‌پذیری و برچسب.
- ارزیابی موتور شواهد قطعی روی محک مصنوعی جفتی پایتون و تابع‌های واقعی دیویگن، به‌همراه مقایسه رودرو با بندیت و فلافایندر.

^۸Label Leakage

- تحلیل نتایج و استخراج این نتیجه‌گیری که اثربخشی بازبینی مبتنی بر شواهد به کیفیت و دانه‌بندی شواهد ارائه‌شده وابسته است.

۱.۵ روش پیشنهادی

روش پیشنهادی این پژوهش، بر پایه این مشاهده استوار است که تحلیل ایستا و مدل زبانی، نقطه قوت مکمل دارند. ابزار ایستا نامزدهای بی‌شمار و ارزان تولید می‌کند، اما در قضاوت نهایی ضعیف است؛ مدل زبانی در فهم زمینه اجرا توانمند است، اما به‌تنهایی نامزد قابل‌اتکایی پیش نمی‌دهد. بر این اساس، در سامانه پیشنهادی، ابزارهای ایستا تنها به‌عنوان تأمین‌کننده نامزد به کار می‌روند و تصمیم نهایی بر عهده دو عامل مستقل است: عامل جستجوگر فرضیه‌های آسیب‌پذیری را پیشنهاد می‌کند و عامل ارزیاب هر فرضیه را جداگانه، تنها بر پایه بسته شواهد ارائه‌شده، تأیید یا رد می‌کند.

برای آنکه داوری عامل‌های زبانی قابل اعتماد باشد، سه تصمیم طراحی اتخاذ شده است. نخست، شواهد به‌صورت ساخت‌یافته و با کمترین زمینه اضافی ارائه می‌شود تا هم حجم توکن کاهش یابد و هم تمرکز مدل بر همان نقطه مورد ادعا بماند. دوم، هر متنی که ممکن است برچسب را افشا کند، پیش از ارسال پاک‌سازی می‌شود. سوم، به‌جای دوگزینه‌ای بودن، سه خروجی تعریف شده است تا عامل بتواند در نبود شاهد کافی، صریحاً عدم قطعیت خود را اعلام کند.

۱.۶ ساختار پایان‌نامه

ساختار این پایان‌نامه به شرح زیر است. در فصل دوم، مبانی نظری شامل تحلیل ایستا، تحلیل ایستا-پویا، مدل‌های زبانی بزرگ و عامل‌های نرم‌افزاری، مجموعه داده جولیت و کارهای پیشین بررسی می‌شود. در فصل سوم، معماری سامانه، ساختار بسته شواهد، سازوکار پاک‌سازی کد، سیاست تطبیق یافته‌ها و معیارهای ارزیابی تشریح می‌شود. در فصل چهارم، طراحی آزمایش‌ها، نتایج به‌دست‌آمده، تحلیل به‌ازای نوع آسیب‌پذیری، بررسی نمونه‌های خطا و تهدیدهای روایی ارائه می‌شود. در فصل پنجم، نتیجه‌گیری، محدودیت‌ها و کارهای آینده بیان می‌شود.

فصل دوم

مبانی نظری و کارهای پیشین

۲.۱ تحلیل ایستا و مفهوم هشدار نادرست

تحلیل ایستا مجموعه‌ای از روش‌های خودکار است که هدف آن، یافتن خطاها و نقض‌های قرارداد در برنامه بدون اجرای آن است. این روش‌ها بر پایه ساخت یک نمایش میانی از برنامه، مانند گراف کنترل جریان یا درخت نحو انتزاعی، و سپس اعمال قواعد تحلیل روی آن نمایش استوارند. دو خانواده اصلی از تحلیل ایستا عبارت‌اند از: تحلیل مبتنی بر قواعد^۱ که با الگوهای از پیش تعریف‌شده الگوهای بد، مانند استفاده از فراخوانی‌های خطرناک، کار می‌کند؛ و تحلیل مبتنی بر تفسیر انتزاعی^۲ که با تقریب زدن مجموعه مقادیر ممکن یک متغیر در هر نقطه برنامه، به صورت ایستا به رفتار برنامه نزدیک می‌شود. این دو خانواده، مبنای تقریباً همه ابزارهای تجاری و متن‌باز امروزی هستند.

هر ابزار تحلیل ایستا زمانی ارزشمند است که بتواند میان سه دسته هشدار تمایز بگذارد: هشدار درست، هشدار نادرست و از دست رفتن آسیب‌پذیری واقعی. هشدار درست یعنی محل گزارش‌شده واقعاً یک نقص امنیتی دارد. هشدار نادرست یعنی محل گزارش‌شده ایرادی ندارد؛ برای نمونه مسیر داده هشدار داده‌شده هرگز اجرا نمی‌شود، یا اندازه بافر پیش از استفاده کنترل شده است. از دست رفتن آسیب‌پذیری واقعی یعنی ابزار ساکت مانده است، در حالی که کد آسیب‌پذیر است. اندازه‌گیری صرفاً دقت و یادآوری، بدون توجه به این‌که این معیارها با چه دانه‌بندی و با چه سیاستی محاسبه شده‌اند، مقایسه ابزارها را گمراه‌کننده می‌کند؛ از این رو در فصل چهارم دانه‌بندی معیارها به صراحت تعریف و در همه آزمایش‌ها به طور یکسان اعمال می‌شود. بررسی‌های پیشین نشان می‌دهد که هشدار نادرست و شیوه ارائه هشدارها از موانع اصلی به کارگیری ابزارهای تحلیل ایستا هستند [۶]. همچنین تجربه صنعتی نشان داده است که بررسی دستی هزاران هشدار، هزینه قابل توجهی دارد و اعتماد توسعه‌دهندگان به ابزار را کاهش می‌دهد [۷].

۲.۲ ابزارهای تحلیل ایستای به کاررفته در این پژوهش

در فاز یک سامانه پیشنهادی، برای هر زبان، ابزارهای متن‌باز مناسب به صورت موازی روی هر فایل اجرا می‌شوند. برای پایتون از بندیت و برای C و C++ از سه ابزار استفاده می‌شود. انتخاب این ابزارها بر پایه تنوع رویکرد و پوشش زبان‌های دامنه پژوهش انجام شده است: بندیت الگوهای امنیتی پایتون را پوشش می‌دهد؛ یکی مبتنی بر قواعد ساده و سریع، یکی مبتنی بر الگوهای خطرناک توابع، و یکی مبتنی بر تحلیل مسیر و وابستگی به سامانه ساخت است.

ابزار	رویکرد	نقطه قوت	نقطه ضعف
cppcheck	قاعده‌محور	سریع و کم‌هزینه	عمق تحلیل کم
flawfinder	قاعده‌محور	پوشش سبک‌های خطرناک	بسیار پرهشدار

^۱ Rule-based

^۲ Abstract Interpretation

ابزار	رویکرد	نقطه قوت	نقطه ضعف
clang	تحلیل مسیر	مسیرپیمایی واقعی	وابسته به کامپایل
bandit	الگوی امنیتی پایتون	پوشش دامنه پایتون	هشدار الگویی

جدول ۲.۱: ویژگی ابزارهای تحلیل ایستای به کاررفته در فاز یک

مزیت اجرای موازی ابزارهای هر زبان، پوشش بیشتر و کاهش وابستگی به نقاط کور یک ابزار واحد است. عیب آن، تولید هشدارهای همپوشان است؛ به عنوان مثال، ممکن است چند ابزار یک سرریز بافر واحد را با چند شناسه متفاوت گزارش کنند. این هشدارهای تکراری، اگر تفکیک نشوند، هم آمار هشدار نادرست را بدتر نشان می دهند و هم بار کاری عامل جستجوگر را چند برابر می کنند. به همین دلیل، در فاز یک، یافته های همپوشان با استفاده از اثر انگشت و تطبیق خط در یک گروه ادغام می شوند.

۲.۳ تحلیل ایستا-پویا و رویکردهای ترکیبی

تحلیل پویا، برخلاف تحلیل ایستا، برنامه را اجرا می کند و رفتار واقعی آن را مشاهده می نماید. این رویکرد دقت بالایی برای خطاهایی دارد که در مسیر اجرایی رخ می دهند، اما تنها بخش کوچکی از کد را پوشش می دهد، زیرا اجرای همه حالت های ممکن یک برنامه، به طور کلی ناممکن است. رویکردهای ترکیبی^۳ تلاش می کنند این دو مزیت را همزمان به دست آورند؛ برای نمونه، با استفاده از تحلیل ایستا برای یافتن محل های مشکوک و سپس اجرای هدفمند برنامه در همان محل ها. سامانه پیشنهادی این پژوهش نیز در همین خانواده مفهومی قرار می گیرد، با این تفاوت که به جای تحلیل پویا، از دو عامل زبانی مستقل به عنوان پیشنهاددهنده و داور مبتنی بر شواهد استفاده می کند.

۲.۴ مدل های زبانی بزرگ و عامل های نرم افزاری

مدل های زبانی بزرگ، مدل های آماری زبانی با توانایی تولید متن بر اساس پیش بینی توکن بعدی هستند که با مقیاس دهی حجم داده و پارامتر آموزش داده می شوند. این مدل ها در چند سال اخیر توانسته اند در وظایفی مانند تولید کد، خلاصه سازی و تحلیل ایستا مورد استفاده قرار گیرند [۱۱].

عامل نرم افزاری^۴ به برنامه ای اطلاق می شود که با دریافت یک هدف، به طور خودکار برنامه ریزی می کند، ابزار فراخوانی می کند و تا رسیدن به پاسخ، چرخه مشاهده-تصمیم-اقدام را تکرار می کند. در این پژوهش، عامل با یک نقش محدود تعریف شده است: پیشنهاد یک یافته مشخص یا داوری درباره همان یافته بر پایه بسته شواهد ارائه شده. این محدودیت عامدانه است، زیرا جستجوگر باید در برابر وسوسه اثبات یافته مقاوم باشد و ارزیاب باید در برابر وسوسه تولید یافته جدید مقاوم باشد.

^۳ hybrid analysis^۴ Software Agent

۲.۵ استدلال مبتنی بر شواهد

یکی از محدودیت‌های شناخته‌شده مدل‌های زبانی این است که می‌توانند پاسخی روان، مرتبط و در ظاهر مستند تولید کنند، در حالی که هیچ‌یک از اجزای آن پاسخ در متن ورودی موجود نبوده است. این پدیده که ریشه در نبود پیوند میان توکن‌های تولیدشده و جهان واقعی دارد، در ادبیات هوش مصنوعی با مسئله خاک افتادگی^۵ توصیف شده است [۱۴].

راهکاری که در این پژوهش به کار گرفته شده، محدودکردن عامل به داوری بر پایه شواهد است. عامل موظف است تنها از محتوای بسته شواهد استفاده کند، در صورت کمبود شاهد، عدم قطعیت را اعلام نماید و به جای استنتاج آزاد، به قطعه کد و ساختار ارائه‌شده استناد کند. این رویکرد با خانواده روش‌های بازیابی‌شده^۶ هم‌خانواده است، با این تفاوت که هدف، افزایش دانش مدل نیست، بلکه مهار آن و کاهش پاسخ‌های بی‌پایه است [۱۳].

۲.۶ مجموعه داده جولیت

مجموعه آزمون جولیت^۷ یک مجموعه داده مرجع برای ارزیابی ابزارهای تحلیل ایستا است که توسط مؤسسه ملی استاندارد و فناوری آمریکا به عنوان بخشی از پایگاه آزمون ارزیابی امنیت نرم‌افزار منتشر شده است [۴]. این مجموعه برای هر نوع آسیب‌پذیری، جفت‌هایی از نسخه آسیب‌پذیر و نسخه سالم از یک سناریوی واحد تولید می‌کند. تنها تفاوت میان نسخه آسیب‌پذیر و سالم، در نقطه‌ای از جریان داده است؛ برای مثال، در یک سناریوی سرریز عدد صحیح، نسخه سالم همان محاسبه را با یک شرط کنترلی همراه می‌کند.

CWE190_Integer_Overflow__int_rand_add_73b_bad.cpp

```
void badSink(list<int> dataList)
{
    int data = dataList.back();
    {
        /* POTENTIAL FLAW: Adding 1 to data could
           cause an overflow */
        int result = data + 1;
        printIntLine(result);
    }
}
```

شکل ۲.۱: نسخه آسیب‌پذیر یک سناریوی جولیت

^۵ Grounding

^۶ Retrieval-Augmented Generation

^۷ Juliet Test Suite

CWE190_Integer_Overflow__int_rand_add_73b_good.cpp

```

void goodB2GSink(list<int> dataList)
{
    int data = dataList.back();
    /* FIX: Add a check to prevent an overflow */
    if (data < INT_MAX) {
        int result = data + 1;
        printIntLine(result);
    }
}

```

شکل ۲.۲: نسخه سالم همان سناریو؛ تنها تفاوت، یک شرط کنترلی است

ویژگی جفت‌بودن نسخه‌ها، هم فرصت و هم تهدید است. فرصت از آن رو که هر دو نسخه زمینه تقریباً یکسانی دارند، بنابراین یک هشدار نادرست را می‌توان به‌سادگی به توضیحی بیرونی نسبت داد. تهدید از آن رو که اگر مرزبندی داده‌ها درست انجام نشود، نسخه‌های مرتبط یک سناریو می‌توانند در دو بخش آموزش و آزمون قرار گیرند و نتیجه ارزیابی به‌شدت خوش‌بینانه شود. به همین دلیل، در این پژوهش برای هر نمونه یک شناسه گروه تعریف شده است تا نمونه‌های هم‌خانواده در یک بخش قرار گیرند.

علاوه بر جولیت، دو مجموعه دیگر برای پوشش زبان پایتون و کد واقعی به کار رفت. نخست، یک محک مصنوعی جفتی شامل ۴۸ فایل پایتون است؛ ۲۴ سناریوی آسیب‌پذیر و نسخه اصلاح‌شده همان سناریو. این محک برای سنجش مهار هشدار نادرست ساخته شده و جایگزین مجموعه واقعی نیست. دوم، مجموعه دیویگن است که در نسخه به‌کاررفته شامل ۲۷۲۵۸ تابع واقعی و برچسب‌دار C از پروژه‌های متن‌باز است؛ برخلاف توصیف خلاصه پیشنهاد، این نسخه صرفاً توابع کرنل لینوکس نیست. خوانش برچسب‌های سارد و بیگ‌وال در پیاده‌سازی پشتیبانی می‌شود، اما چون نسخه معتبر و قابل بازیابی آن‌ها در این پژوهش در دسترس نبود، عددی از آن‌ها گزارش نمی‌شود.

۲.۷ کارهای پیشین

کارهای پیشین را می‌توان در سه دسته دید. دسته نخست، ارزیابی‌های انتقادی ابزارهای تحلیل ایستا هستند که نشان می‌دهند موانعی مانند هشدار نادرست و شیوه ارائه هشدار، پذیرش این ابزارها را محدود می‌کند [۶] [۷] و تحلیل ایستای سبک می‌تواند خطاهای پیاده‌سازی شناخته‌شده را پیش از استقرار بیابد [۸]. دسته دوم، روش‌های یادگیری ماشین برای تشخیص آسیب‌پذیری هستند؛ از جمله مدل دیویگن که با یادگیری معنای برنامه روی تابع‌های واقعی زبان C بهبود چشمگیری نسبت به روش‌های پیشین گزارش کرده است [۹]؛ با این حال، این روش‌ها معمولاً مبانی نظری روشنی برای توضیح دلیل هر هشدار ارائه نمی‌کنند و در برابر کد ناآشنا به‌سختی تعمیم می‌یابند. دسته سوم، کاربرد مدل‌های زبانی بزرگ در تحلیل کد است؛ از جمله بررسی سهم کد تولیدشده توسط دستیارهای برنامه‌نویسی از نظر آسیب‌پذیری [۱۰] [۱۱] که نشان می‌دهد توانایی تولید کد، به‌تنهایی تضمینی برای ایمنی کد نیست.

دسته	نمونه	نقش	محدودیت	جایگاه این کار
ارزیابی انتقادی	[۶] [۸]	سنجش واقع‌بینانه	وابسته به مجموعه داده	مبانی تعریف معیار
یادگیری ماشین	[۹]	دقت بالاتر	بدون توضیح‌پذیری	مقایسه عددی
مدل زبانی	[۱۰] [۱۱]	فهم زبان طبیعی	توهم و عدم قطعیت	تلفیق با ایستا

جدول ۲.۲: جایگاه کار پیشنهادی در میان کارهای پیشین

۲.۸ جایگاه کار پیشنهادی

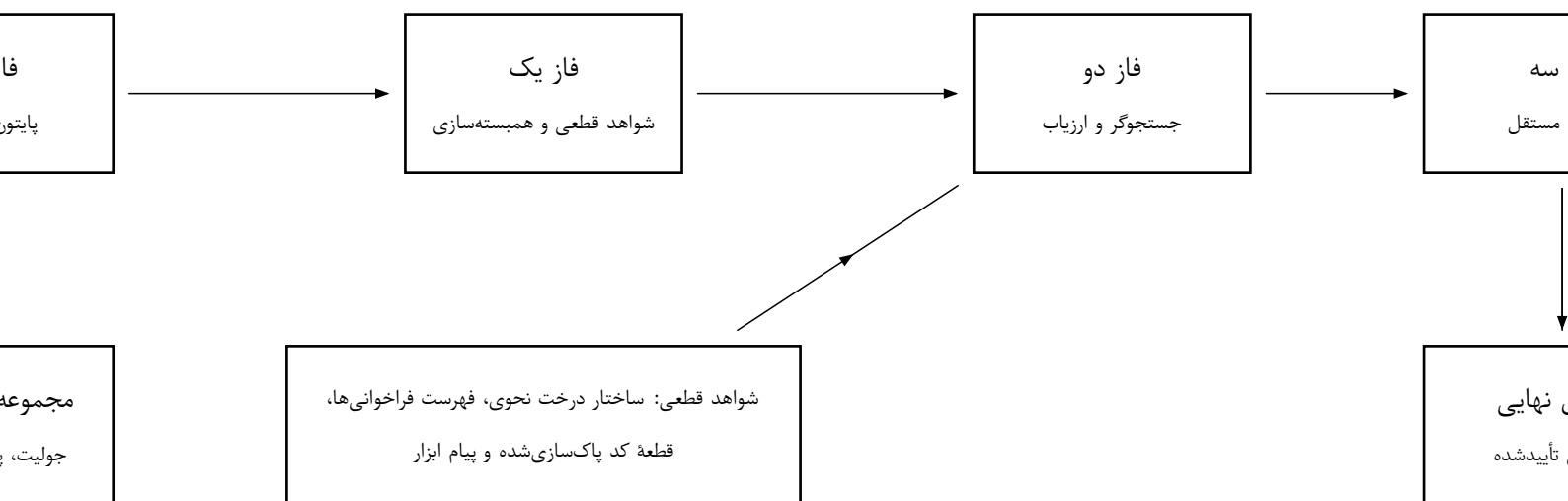
کار پیشنهادی این پژوهش از دسته سوم متأثر است، اما بر یک نکته روش‌شناختی تأکید دارد که در بسیاری از کارهای موجود رعایت نمی‌شود: ارزیابی کمی باید با تعریف صریح معیارها و کنترل نشت برچسب همراه باشد. همچنین، به جای جایگزینی ابزارهای ایستا، این کار آن‌ها را به عنوان تأمین‌کننده نامزد به کار می‌گیرد و از عامل‌های زبانی تنها برای پیشنهاد و پالایش نامزدها بهره می‌گیرد. این انتخاب، امکان گزارش یک نقطه کاری قابل تنظیم را فراهم می‌کند: سامانه می‌تواند با تغییر آستانه تأیید، روی نقطه‌ای از منحنی مبادله دقت و یادآوری بنشیند که برای هر سازمان مناسب‌تر است.

فصل سوم

طراحی و پیاده‌سازی سامانه

۳.۱ معماری کلی

سامانه پیشنهادی از سه فاز متوالی تشکیل شده است. فاز یک، شواهد قطعی است و مسئولیت آن تولید فهرستی از یافته‌های خام ابزارها، استخراج ساختار نحوی، بازیابی مسیر منبع تا نقطه مصرف و همبسته‌سازی یافته‌های تکراری است. فاز دو، داوری چندعامله است: عامل جستجوگر فرضیه‌های آسیب‌پذیری را پیشنهاد می‌کند و عامل ارزیاب هر فرضیه را جداگانه بررسی و تأیید یا رد می‌کند. فاز سه، ارزیابی مستقل است و پیش‌بینی‌ها را با همان سیاست تطبیق و معیارها در برابر حقیقت زمین می‌سجد. این تفکیک فازها عمدی است: هر فاز یک پرسش مشخص را پاسخ می‌دهد و خروجی آن قابل ارزیابی جداگانه است، بنابراین می‌توان اثر هر فاز را مستقل از دو فاز دیگر سنجید.



شکل ۳.۱: معماری سه‌فاز سامانه و جریان داده میان فازها

جداسازی فازها یک مزیت عملی نیز دارد. اجرای ابزارهای ایستا هزینه پردازشی محلی دارد، در حالی که فراخوانی مدل زبانی هزینه زمانی و شبکه‌ای دارد. جدا بودن این دو، امکان اجرای موازی آن‌ها را روی مجموعه‌ای از نخ‌ها و یک زمان‌بند پرتاب^۱ می‌دهد، بدون آنکه فراخوانی‌های زبانی در انتظار، اجرای ابزارهای ایستا را متوقف کنند.

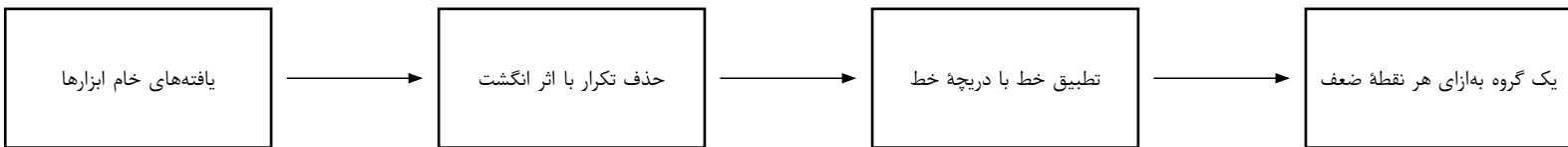
۳.۲ فاز یک: تحلیل ایستا

در فاز یک، برای هر فایل، ابزارهای متن‌باز متناسب با زبان آن اجرا می‌شوند: بندیت برای پایتون و سی‌پی‌پی‌چک، فلاپایندر و تحلیل‌گر ایستای کلنگ برای C و C++. خروجی خام هر ابزار به فهرستی ساخت‌یافته از یافته‌ها تبدیل می‌شود. برای هر یافته، شناسه ابزار، شماره خط، شدت، پیام و در صورت وجود، شناسه CWE ذخیره می‌شود. انتخاب چند ابزار به جای یک ابزار، به دلیل پوشش مکمل آن‌هاست: هیچ‌یک به تنهایی همه سناریوهای مجموعه ارزیابی را پوشش نمی‌دهد. هم‌زمان، تجزیه‌گر درخت نحوی [۱۲] توابع، فراخوانی‌ها، انتساب‌ها، ارجاع‌ها و واردسازی‌ها را استخراج می‌کند و موتور بازیابی مسیر، مسیرهای نامزد منبع تا نقطه مصرف را در محدوده همان تابع می‌سازد.

^۱ Scheduler

۳.۳ فاز یک: همبسته‌سازی یافته‌ها

خروجی خام ابزارها سه مشکل دارد که پیش از بازبینی باید برطرف شود. نخست، یافته‌های تکراری است؛ چند ابزار ممکن است یک نقص واحد را گزارش کنند. دوم، گزارش‌های ناهمگون است؛ پیام‌ها قالب یکسانی ندارند و برخی تنها یک قاعده سبک را گزارش می‌کنند. سوم، یافته‌های پراکنده‌اند؛ یک نقطه ضعف واحد ممکن است در چند خط و با چند شناسه متفاوت ظاهر شود.



شکل ۳.۴: زنجیره ادغام یافته‌های تکراری و تشکیل گروه‌های بازبینی

برای رفع این سه مشکل، سه گام متوالی تعریف شده است. گام نخست، حذف تکرار با استفاده از یک اثر انگشت شامل نام فایل، نوع آسیب‌پذیری و خط است. گام دوم، تطبیق خط با یک درجه خط است، به این معنا که دو یافته در فاصله کمتر از پنج خط از یکدیگر، مربوط به یک نقطه ضعف واحد تلقی می‌شوند. گام سوم، ساخت گروه بازبینی است؛ هر گروه، مجموعه یافته‌های ادغام‌شده به‌همراه فهرست ابزارهای گزارش‌دهنده آن است. هدف از این ادغام، کاهش بار کاری عامل جستجوگر و جلوگیری از چند بار پرسش درباره یک نقطه ضعف واحد است.

۳.۴ فاز دو: داوری چندعامله

در فاز دو، عامل جستجوگر ابتدا همه فایل را می‌بیند و فهرستی از فرضیه‌های آسیب‌پذیری، شامل شناسه مشکوک CWE، خط مورد بازبینی و توصیف یک‌جمله‌ای منبع و نقطه مصرف، تولید می‌کند. سپس عامل ارزیاب هر فرضیه را جداگانه بررسی می‌کند و باید یکی از سه تصمیم جدول ۳.۱ را برگرداند. دو نکته در این طراحی قابل تأمل است. نخست، وجود گزینه نامطمئن، عامل را از دوگزینه‌ای بودن رها می‌کند و اجازه می‌دهد در نبود شاهد کافی، صریحاً عدم قطعیت خود را اعلام نماید؛ خروجی نامطمئن در گزارش نهایی به‌عنوان نیازمند بررسی انسانی حفظ می‌شود. دوم، ارزیاب استدلال جستجوگر را نمی‌بیند و تنها فرضیه را دریافت می‌کند؛ این تفکیک، اثر لنگرانداختن استدلال اولیه بر داوری نهایی را کاهش می‌دهد. تنها خروجی ارزیاب می‌تواند به یافته نهایی تبدیل شود و همه تأییدها، رد‌ها و موارد نامطمئن در گزارش تصمیم ثبت می‌شوند.

خروجی	معنا	اثر بر گزارش
تأییدشده	CONFIRMED	شاهد کافی برای پذیرش یافته
ردشده	REJECTED	شاهد ناسازگار با یافته
نامطمئن	UNCERTAIN	شاهد ناکافی
		نیازمند بررسی انسانی

جدول ۳.۱: سه خروجی ممکن عامل ارزیاب

۳.۵ بسته شواهد

کیفیت تصمیم عامل، تابع کیفیت شواهدی است که در اختیارش قرار می‌گیرد. بسته جستجوگر شامل نقش، زبان، فایل، زمینه کد، یافته‌های ابزارهای ایستا، خلاصه ساختاری و مسیرهای نامزد منبع تا نقطه مصرف است. بسته ارزیاب برای هر فرضیه شامل همان فرضیه، زمینه کد پیرامون خط مشکوک، مسیرهای نزدیک به آن فرضیه و یافته‌های نزدیک ابزارهاست؛ اما استدلال جستجوگر در آن نیست. موتور قطعی، مسیر منبع تا نقطه مصرف را در محدوده همان تابع بازیابی می‌کند و برای هر مسیر، عبارت منبع، عبارت نقطه مصرف، شماره خطها، گام‌های میانی، منشأ آلودگی و در صورت وجود، قانون کاهش خطر را ثبت می‌نماید.

evidence-packet.json

```
{
  "role": "scanner",
  "language": "python",
  "file": "example.py",
  "source_context": {
    "start_line": 4,
    "end_line": 12,
    "snippet": "    6: name = sys.argv[1]\n ..."
  },
  "static_tool_findings": [
    { "tool": "bandit", "message": "possible shell injection" }
  ],
  "structural": {
    "functions": [ { "name": "run_user_command", "line": 4 } ],
    "calls": [ { "callee": "subprocess.call", "line": 6 } ]
  },
  "dataflow_chains": [
    {
      "source": "sys.argv[1]",
      "sink": "subprocess.call(command, shell=True)",
      "mitigated": false
    }
  ]
}
```

شکل ۳.۲: نمونه ساده‌شده بسته شواهد جستجوگر

قطعه کد با شعاع هشت خط پیش و پس از خط مورد نظر ساخته می‌شود. این مقدار، مصالحه‌ای است میان هزینه توکن و کیفیت زمینه: شعاع کمتر، تعریف متغیر یا نقطه کنترلی مرتبط را از قطعه بیرون می‌گذارد و شعاع بیشتر، توجه مدل را پراکنده می‌کند. شواهد ساختاری با استفاده از تجزیه‌گر درخت نحوی تولید می‌شود و به عامل امکان می‌دهد به جای حدس زدن، فهرست فراخوانی‌های همان تابع را ببیند. در فصل چهارم نشان داده می‌شود که افزودن این بخش، تعداد تأییدهای عامل را افزایش می‌دهد. اعتماد ارزیاب نیز به صورت قطعی مهار می‌شود: تأیید کم اعتماد تنزل می‌یابد، ادعای تأیید زنجیره باید با خود رأی ارزیاب سازگار باشد، و برای دسته‌هایی که موتور مدل می‌کند، نبود مسیر واقعی در فایل به رد قطعی می‌انجامد.

۳.۶ پاک‌سازی کد و جلوگیری از نشت برچسب

مجموعه داده جولیت برچسب واقعی هر فایل را درون خود متن فایل قرار می‌دهد. برای نمونه، در فایل‌های آسیب‌پذیر، حاشیه‌نویسی‌هایی با عبارت‌هایی نظیر نشانگر وجود نقص و شماره CWE وجود دارد و نام فایل و نام اشیا نیز شامل واژه بد یا خوب هستند. اگر این متن عیناً به مدل داده شود، مدل می‌تواند بدون هیچ تحلیلی پاسخ درست را از روی نام فایل حدس بزند و نتیجه آزمایش، اندازه‌گیری توانایی تحلیل نخواهد بود، بلکه اندازه‌گیری توانایی خواندن برچسب. برای جلوگیری از این نشت، سه اقدام متوالی در ماژول پاک‌سازی انجام می‌شود. نخست، محتوای حاشیه‌نویسی‌ها در جای خود با فاصله جایگزین می‌شود، به گونه‌ای که تعداد خط‌ها و شماره خط گزارش شده توسط ابزارها تغییر نکند. دوم، شناسه‌های افشاگر، مانند نام‌هایی که شامل واژه بد یا خوب هستند، به نام‌هایی پایدار و بی‌معنا نگاشت می‌شوند؛ به این ترتیب که یک نام در قطعه کد، در شواهد ساختاری و در پیام ابزار، همیشه به یک شکل ظاهر شود. سوم، پاک‌سازی روی تمام فیلدهای متنی بسته شواهد اعمال می‌شود، نه فقط بر قطعه کد، زیرا پیام ابزارها و نام فایل نیز می‌توانند برچسب را افشا کنند.

CWE122_Heap_Based_Buffer_Overflow__c_CWE805_struct_memcpy_82a_bad.cpp

```

24:
25: void sym_1902e3()
26: {
27:     twoIntsStruct * data;
28:     data = NULL;
29:
30:     data = (twoIntsStruct *)malloc(50*sizeof(twoIntsStruct));
31:     if (data == NULL) {exit(-1);}
32:     sym_9623a0* baseObject = new sym_5b7d8f;
33:     baseObject->action(data);
34:     delete baseObject;
35: }
```

شکل ۳.۳: قطعه کد پس از پاک‌سازی؛ حاشیه‌نویسی‌ها محو و شناسه‌های افشاگر بازنام‌گذاری شده‌اند

یک استثنا عمدی است: ارجاع‌هایی که شکل CWE-122 دارند بازنویسی نمی‌شوند، زیرا سازوکار تطبیق یافته‌ها به آن‌ها نیاز دارد. همچنین، برای اطمینان از پایداری رفتار سامانه، آزمون‌های خودکاری نوشته شده است که در صورت بازگشت نشت برچسب، آزمایش شکست می‌خورد؛ زیرا در آن صورت همه آزمایش‌های مبتنی بر مدل زبانی به‌طور خاموش به سنجش نشت تبدیل می‌شوند.

۳.۷ سیاست تطبیق یافته‌ها و معیارها

سنجش خروجی سامانه، نیازمند تصمیم درباره این است که یک هشدار چه زمانی با یک آسیب‌پذیری واقعی منطبق دانسته شود. در این سامانه، سه قانون به کار می‌رود که در جدول ۳.۲ خلاصه شده‌اند.

قانون تطبیق	شرط	دلیل
هم‌بودن فایل	نام فایل یکسان	تطبیق در سطح پروژۀ واقعی
خانواده CWE	هم‌پوشانی خانواده	ابزارها CWE والد گزارش می‌کنند
فاصله خط	کمتر از ۵ خط	خطای گزارش ابزار جبران می‌شود

جدول ۳.۲: سیاست تطبیق یافته‌ها با برچسب واقعی

نخستین قانون، الزام هم‌بودن فایل است؛ بدون آن، هیچ تطبیقی پذیرفته نمی‌شود. دومین قانون، استفاده از خانواده CWE است. این قانون از یک واقعیت عملی ناشی می‌شود: ابزارها اغلب شناسۀ والد را گزارش می‌کنند، در حالی که برچسب مجموعه‌داده فرزند را نام می‌برد؛ برای نمونه، یک ابزار ممکن است سرریز بافر کلاسیک را گزارش کند، در حالی که نمونه متعلق به سرریز بافر مبتنی بر حافظۀ چرخه‌ای است. بدون نگاشت خانواده، این یافته‌های واقعی، هم‌زمان هشدار نادرست و از دست رفتن آسیب‌پذیری شمرده می‌شوند. سومین قانون، تحمل پنج خطی فاصله خط است.

در مورد معیارها، یک تصمیم روش‌شناختی مهم گرفته شده است. مثبت درست، مثبت نادرست و منفی نادرست در دانه‌بندی یافته محاسبه می‌شوند، یعنی هر هشدار جداگانه شمرده می‌شود. منفی درست اما در دانه‌بندی فایل محاسبه می‌شود، یعنی یک فایل سالم تنها زمانی منفی درست است که هیچ هشدار نداشته باشد. دلیل این انتخاب آن است که هشدار نادرست، پدیده‌ای در سطح فایل است؛ توسعه‌دهنده با فایلی روبه‌رو می‌شود که هشدار دارد، نه با یک خط منفرد. برای رفع ابهام، نرخ هشدار روی فایل سالم به‌طور جداگانه گزارش می‌شود.

۳.۹ شمای گزارش و نمونه‌های واقعی

خروجی سامانه یک فایل گزارش در قالب JSON است و هر یافته تأییدشده باید همه فیلدهای جدول ۳.۳ را داشته باشد. مرجع CVE از فهرست دستی کنترل‌شده پر می‌شود، نه از خروجی مدل؛ بنابراین شناسۀ جعلی نمی‌تواند وارد گزارش

شود. اگر برای یک دسته نمونه واقعی منتخب وجود نداشته باشد، گزارش تنها پیوند صفحه کلاس CWE را می‌آورد و جای خالی را با حدس پر نمی‌کند.

فیلد گزارش	محتوا	الزام پیشنهاد
cwe	شناسه دسته ضعف	نوع آسیب‌پذیری
severity	شدت	تحلیل گزارش
file, line, function	فایل، خط و تابع	محل دقیق کد
explanation	توضیح فنی کوتاه	توضیح تحلیلی
source, sink, chain	عبارت‌ها، خط‌ها و مسیر بازیابی‌شده	شاهد جریان داده
chain_verified, taint_origin	تأیید زنجیره و منشأ آلودگی	شاهد جریان داده
related_cves, references	شناسه واقعی و پیوند مرجع	مرجع CVE
confidence, detected_by	اعتماد و نقش عامل‌ها	قابلیت حساسی

جدول ۳.۳: شمای گزارش نهایی سامانه

جدول ۳.۴ دامنه پیشنهاد را با نمونه‌های واقعی پیاده‌سازی مقایسه می‌کند. دو شناسه پیشنهاد نادرست بودند و در پیاده‌سازی اصلاح شدند: به جای CVE-2021-25239 که یک مسئله افشای اطلاعات در محصول ترندمیکرو است، نمونه تزریق قالب جینجا CVE-2025-27516 به کار رفت؛ و چون CVE-2022-22817 تزریق عبارت در کتابخانه پیلو است نه بازسازی ناامن در پای‌یامل، نمونه تأییدشده CVE-2017-18342 برای بازسازی ناامن افزوده شد. شناسه نادرست پیشنهاد در فهرست مرجع به صورت خارج از دامنه و با دلیل نگه داشته شده است تا حذف آن پنهان نباشد. برای CVE-2021-3156، CVE-2021-18342، CVE-2017-7529، CVE-2022-22817، CVE-2025-27516 و CVE-2021-25239 به ترتیب به مراجع [۱۶] تا [۲۱] مراجعه شود.

دسته	زبان	نمونه واقعی پیاده‌سازی	نسبت با پیشنهاد
CWE-122	C	CVE-2021-3156	همان نمونه پیشنهاد؛ مرجع ردهت CWE-122 و علت ریشه‌ای CWE-193 است
CWE-190	C	CVE-2017-7529	همان نمونه پیشنهاد
CWE-94	Python	CVE-2022-22817	همان شناسه پیشنهاد، ولی نمونه واقعی تزریق عبارت در پیلو است، نه بازسازی ناامن در پای‌یامل
CWE-1336	Python	CVE-2025-27516	جایگزین شناسه نادرست پیشنهاد برای تزریق قالب

دسته	زبان	نمونه واقعی پیاده‌سازی	نسبت با پیشنهاد
CWE-502	Python	CVE-2017-18342	نمونه تأییدشده بازسازی ناامن در پای‌یامل؛ شناسه پیشنهاد متعلق به آن نیست
CWE-78	Python	بدون شناسه منتخب	تنها مرجع کلاس؛ همان دامنه پیشنهاد

جدول ۳.۴: دامنه پیشنهاد و نمونه‌های واقعی به‌کاررفته در گزارش

۳.۱۰ زیرساخت اجرا و تکرارپذیری

یکی از پیش‌نیازهای هر ادعای کمی، تکرارپذیری است. به همین دلیل، هر اجرای سامانه، در کنار نتایج، فراداده‌ای شامل نسخه هر ابزار، نام و نسخه مدل زبانی، نشانی سرویس، شعاع قطعه کد، تحمل خط، سیاست تطبیق CWE و مدت اجرا را در کنار نتایج ذخیره می‌کند. همچنین، اجرای بازبینی به‌گونه‌ای پیاده‌سازی شده است که در صورت قطع پاسخ سرویس، اجرای دوباره تنها موارد ناموفق را پردازش کند. این ویژگی، هم خطای شبکه را از خطای روش‌شناختی جدا می‌کند و هم هزینه ارزیابی تکراری را کاهش می‌دهد.

فصل چهارم

ارزیابی تجربی

۴.۱ پرسش‌های پژوهش

ارزیابی تجربی این پژوهش بر پنج پرسش استوار است. پرسش نخست: بازبینی مبتنی بر شواهد، آیا هشدارهای نادرست تحلیل ایستا را کاهش می‌دهد؟ پرسش دوم: بهای این کاهش از نظر یادآوری چقدر است؟ پرسش سوم: افزودن شواهد ساختاری، اثر بازبینی را بهبود می‌دهد؟ پرسش چهارم: در مقایسه با هر یک از دو مؤلفه به‌تنهایی، ترکیب آن‌ها چه تصویری از منحنی مبادله دقت و یادآوری به دست می‌دهد؟ پرسش پنجم: موتور قطعی بازبینی مسیر، بدون اتکا به مدل زبانی، روی زبان پایتون و کد C و C++ چه دقت، یادآوری و نرخ هشدار دارد؟

۴.۲ مجموعه داده و آماده‌سازی آن

برای آنکه هر چهار پیکربندی روی یک جمعیت یکسان سنجیده شوند، از مجموعه داده جولیت یک زیرمجموعه متعادل و لایه‌بندی شده ساخته شد. این زیرمجموعه شامل ۶۰۰ فایل است: ۳۰۰ فایل آسیب‌پذیر و ۳۰۰ فایل سالم. لایه‌بندی بر پایه نوع آسیب‌پذیری و برچسب انجام شده است، به‌گونه‌ای که از هر یک از چهار لایه، یعنی ترکیب نوع آسیب‌پذیری با برچسب، دقیقاً ۱۵۰ نمونه با بذر ثابت انتخاب شود.

ویژگی	مقدار	توضیح
زبان‌ها	C, C++	هر دو خانواده کد مدل می‌شوند
آسیب‌پذیری‌ها	CWE-122, CWE-190	سرریز بافر و سرریز عدد صحیح
تعداد نمونه ارزیابی	۶۰۰	۳۰۰ آسیب‌پذیر و ۳۰۰ سالم
لایه‌بندی	۱۵۰ در هر لایه	چهار لایه، با بذر ۴۲
ابزارهای فاز یک	cppcheck 2.22, flawfinder 2.0.20, clang 22	نسخه‌های به‌کاررفته
مدل بازبین	mimo-v2.6-flash-free	از راه سرور محلی

جدول ۴.۱: مشخصات مجموعه داده ارزیابی و پیکربندی آزمایش

برای پرسش پنجم، دو مجموعه مستقل از جولیت به کار رفت. نخست، یک محک مصنوعی جفتی پایتون شامل ۴۸ فایل است: ۲۴ سناریوی آسیب‌پذیر و نسخه اصلاح‌شده همان سناریو، در شش دسته دامنۀ پژوهش. جفتی بودن نسخه‌ها امکان سنجش مهار هشدار نادرست را می‌دهد، اما این مجموعه جایگزین داده واقعی نیست. دوم، نسخه به‌کاررفته مجموعه دیویگن شامل ۲۷۰۲۵۸ تابع واقعی C است. اعداد بندیت و فلاپایندر در این بخش با همان سیاست نمونه‌به‌نمونه به دست آمده‌اند؛ اگر ابزار خطا بدهد، آن نمونه از شمارش پایه و اجتماع کنار گذاشته می‌شود و شمار نمونه‌های نامعتبر گزارش می‌گردد.

علت استفاده از زیرمجموعه به‌جای کل مجموعه داده، دوگانه بود. از یک سو، مقایسه چهار پیکربندی نیازمند جمعیت یکسان است. از سوی دیگر، آزمایش چهارم، یعنی بازبینی عامل زبانی روی تک‌تک فایل‌ها، پرهزینه‌ترین بخش کار است؛

هزینه آن تقریباً با تعداد فایل‌ها متناسب است. انتخاب زیرمجموعه لایه‌بندی‌شده، امکان مقایسه معتبر و در عین حال اجرایی‌شدن آزمایش‌ها را فراهم کرد. برای اطمینان از این‌که نتایج فاز یک وابسته به اندازه مجموعه نباشد، آزمایش فاز یک علاوه بر زیرمجموعه، روی کل ۴۰۹۸ فایل نیز اجرا شد که نتایج آن در بخش ۴.۸ گزارش می‌شود.

۴.۳ طراحی آزمایش‌ها

برای پاسخ به پرسش‌های پژوهش، چهار پیکربندی روی همان زیرمجموعه اجرا شد. این طراحی، هر یک از سه مؤلفه سامانه را در کنار دیگری و در کنار ترکیب کامل می‌سنجد. این چهار پیکربندی از مسیر تک‌عامله نگه‌داشته‌شده برای مقایسه استفاده می‌کنند تا نقش شواهد و بازبینی از نقش تفکیک جستجوگر و ارزیاب جدا بماند. پیکربندی نخست، تنها فاز یک را اجرا می‌کند و همه یافته‌های ابزارها را بدون هیچ فیلتری به‌عنوان خروجی سامانه در نظر می‌گیرد؛ این پیکربندی خط پایه ابزارهای تحلیل ایستا را تعریف می‌کند و در ادامه A نامیده می‌شود. پیکربندی دوم، فاز یک را حذف می‌کند و هر فایل را به‌طور کامل در اختیار عامل زبانی قرار می‌دهد؛ این پیکربندی توان عامل زبانی را بدون کمک ابزارهای ایستا می‌سنجد و B نامیده می‌شود. پیکربندی سوم، خروجی ابزارها را در اختیار عامل می‌گذارد، اما بدون شواهد ساختاری، و C نامیده می‌شود. پیکربندی چهارم، شواهد ساختاری را نیز می‌افزاید و D نامیده می‌شود. دو پیکربندی پایانی دقیقاً یکسان‌اند، جز در وجود یا نبود شواهد ساختاری، و همین تمایز، اثر آن شواهد را به‌طور جداگانه قابل اندازه‌گیری می‌کند. پیکربندی‌های E و F و G مدل زبانی ندارند و تنها موتور قطعی ارزیابی مسیر را می‌سنجند: E روی همان زیرمجموعه ۶۰۰ نمونه‌ای جولیت با پایه فلاپایندر، F روی محک ۴۸ نمونه‌ای پایتون با پایه بندیت، و G روی تابع‌های واقعی دیویگن با مقایسه فلاپایندر روی زیرمجموعه‌ای متعادل اجرا شد.

۴.۴ معیارها و سیاست تطبیق

همان‌گونه که در فصل سوم بیان شد، سه معیار در دانه‌بندی یافته و یک معیار در دانه‌بندی فایل محاسبه می‌شود. دقت و یادآوری بر پایه انطباق یافته‌ها با برچسب واقعی و با سیاست تطبیق توصیف‌شده به دست می‌آید و نرخ هشدار روی فایل سالم، سهم فایل‌های سالمی را نشان می‌دهد که دست‌کم یک هشدار دریافت کرده‌اند. گزارش این معیارها به‌همراه ماتریس سردرگمی، امکان بازسازی کامل هر عدد را فراهم می‌کند.

۴.۵ نتایج اصلی

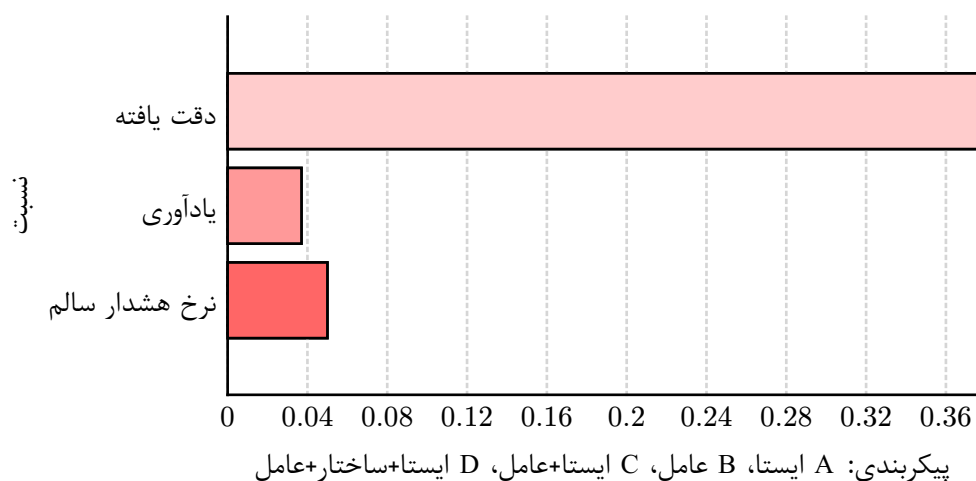
نتایج چهار پیکربندی در جدول ۴.۲ آمده است. نخستین نکته قابل توجه، این است که هیچ پیکربندی برتری مطلق ندارد و هر یک در نقطه‌ای متفاوت از منحنی مبادله دقت و یادآوری می‌نشیند.

آزمایش	پیکربندی	مثبت درست	مثبت نادرست	دقت	یادآوری	F1	نرخ هشدار روی فایل سالم
A	فقط ایستا	۱۱	۱۸	۰٫۳۷۹	۰٫۳۷	۰٫۰۶۷	۰٫۰۵۰

آزمایش	پیکربندی	مثبت درست	مثبت نادرست	دقت	یادآوری	F1	نرخ هشدار روی فایل سالم
B	فقط عامل زبانی	۴۴	۷۵	۰/۳۷۰	۰/۱۴۷	۰/۲۱۰	۰/۱۹۷
C	ایستا + عامل	۳	۲	۰/۶۰۰	۰/۰۱۰	۰/۰۲۰	۰/۰۰۷
D	ایستا + ساختار + عامل	۵	۶	۰/۴۵۵	۰/۰۱۷	۰/۰۳۲	۰/۰۲۰

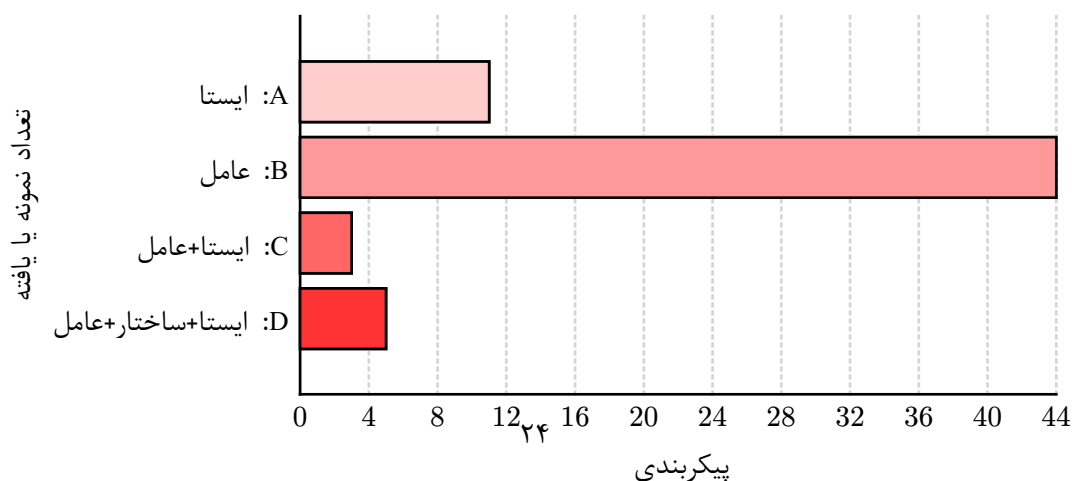
جدول ۴.۲: نتایج اصلی چهار آزمایش روی ۶۰۰ نمونه متعادل

پیکربندی B، یعنی عامل زبانی به تنهایی، بالاترین یادآوری را با ۰/۱۴۷ به دست می‌آورد؛ چهار برابر یادآوری تحلیل ایستا. این یعنی عامل زبانی چیزهایی می‌بیند که ابزارهای ایستا از آن‌ها غافل‌اند. بهای این افزایش یادآوری، بالا رفتن نرخ هشدار روی فایل‌های سالم به ۰/۱۹۷ است، یعنی تقریباً یک فایل سالم از هر پنج فایل، هشدار می‌گیرد. دقت یافته‌ها در این پیکربندی تقریباً با پیکربندی A برابر است، بنابراین افزایش یادآوری با کاهش کیفیت یافته‌ها همراه نبوده است.



شکل ۴.۱: مقایسه دقت، یادآوری و نرخ هشدار روی فایل سالم در چهار پیکربندی

پیکربندی A، یعنی تحلیل ایستا به تنهایی، کمترین نرخ هشدار روی فایل سالم را با ۰/۰۵۰ دارد، اما تنها ۰/۰۳۷ از فایل‌های آسیب‌پذیر را کشف می‌کند. دلیل این یادآوری بسیار پایین، آن است که ابزارهای ایستا با قواعد مبتنی بر الگو، تنها بخش کوچکی از سناریوهای جولیت را فعال می‌کنند. این نتیجه، یک محدودیت مهم را آشکار می‌کند: هر چه سامانه بر پایه تحلیل ایستا بنا شود، یادآوری آن سقفی دارد که تعیین آن، از توان واقعی داوری سامانه مستقل است.



شکل ۴.۲: ماتریس سردرگمی چهار پیکربندی در دانه‌بندی فایل و یافته

پاسخ پرسش نخست پژوهش مثبت است و پاسخ پرسش دوم نیز روشن. بازبینی مبتنی بر شواهد، نرخ هشدار روی فایل سالم را از ۰/۰۵۰ در پیکربندی A به ۰/۰۰۷ در پیکربندی C رساند؛ یعنی حدود ۸۷ درصد کاهش. دقت یافته‌ها نیز از ۰/۳۷۹ به ۰/۶۰۰ افزایش یافت. اما بهای این کاهش، افت یادآوری از ۰/۳۷ به ۰/۱۰ است؛ از یازده مثبت درست در پیکربندی A، تنها سه مورد در پیکربندی C باقی ماند. به بیان دیگر، عامل زبانی علاوه بر نادیده‌گرفتن هشدارهای نادرست، هشت مورد از یافته‌های درست ابزارها را نیز کنار گذاشت. بنابراین سامانه یک کاهش هشدار نادرست به دست می‌آورد، اما این کاهش، کامل و بدون هزینه نیست.

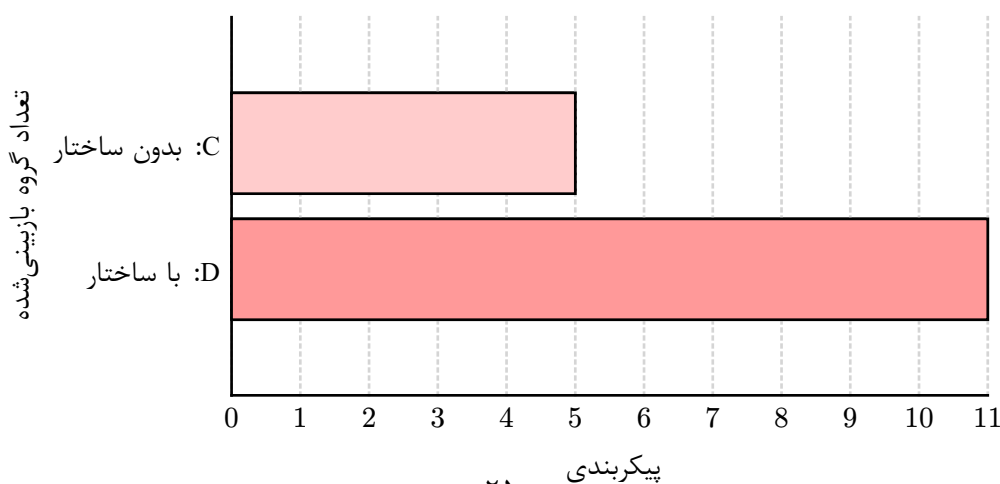
۴.۶ تحلیل قیف تصمیم

برای فهمیدن دلیل افت یادآوری، باید به تصمیم‌های خود عامل نگاه کرد، نه فقط به معیارهای تجمیعی. قیف تصمیم در جدول ۴.۳ آمده است.

آزمایش	تعداد گروه بازبینی	تأییدشده	ردشده / نامطمئن
B	۶۰۰ فایل	۱۱۹	۴ / ۴۷۷
C	۳۱	۵	۱۳ / ۱۳
D	۳۱	۱۱	۶ / ۱۴

جدول ۴.۳: قیف تصمیم عامل زبانی در هر پیکربندی

در پیکربندی C، عامل از ۳۱ گروه، تنها ۵ گروه را تأیید کرد و ۱۳ گروه را رد کرد. بخش قابل توجه، سهم بالای خروجی نامطمئن است؛ ۱۳ گروه از ۳۱ گروه، یعنی حدود ۴۲ درصد، با حالت نامطمئن بسته شدند. این رفتار از آن جهت مطلوب است که عامل به جای داوری قطعی، کمبود شاهد را صادقانه اعلام می‌کند. اما از آن جهت که در خروجی نهایی تنها یافته‌های تأییدشده شمرده می‌شوند، این رفتار به‌طور خودکار به‌معنای از دست رفتن یافته درست است.



شکل ۴.۳: اثر شواهد ساختاری بر تصمیم‌های عامل زبانی

مقایسه دو پیکربندی C و D اثر شواهد ساختاری را روشن می‌کند. با افزودن شواهد ساختاری، تعداد تأییدها از ۵ به ۱۱ رسید و تعداد خروجی‌های نامطمئن از ۱۳ به ۶ کاهش یافت. مثبت‌های درست نیز از ۳ به ۵ افزایش یافت و یادآوری از ۰/۰۱۰ به ۰/۰۱۷ رسید. این نتیجه، پاسخ مثبت پرسش سوم است: شواهد ساختاری، توان عامل را در تصمیم‌گیری افزایش می‌دهد و بخشی از یافته‌هایی را که بدون آن نامطمئن می‌ماند، به تأیید می‌رساند. با این حال، بهای این افزایش، رشد مثبت‌های نادرست از ۲ به ۶ و افزایش نرخ هشدار روی فایل سالم از ۰/۰۰۷ به ۰/۰۲۰ است. شواهد بیشتر، عامل را مطمئن‌تر می‌کند و اطمینان بیشتر در این سامانه، هم‌زمان هم یافته درست و هم یافته نادرست بیشتری تولید می‌کند.

۴.۷ نتایج به تفکیک نوع آسیب‌پذیری

تفکیک نتایج به تفکیک نوع آسیب‌پذیری، تصویری متفاوت و آموزنده ارائه می‌دهد.

آزمایش	نوع	مثبت درست	دقت	یادآوری	F1
A	CWE-122	۱۱	۰/۵۵۰	۰/۰۷۳	۰/۱۲۹
A	CWE-190	۰	۰/۰۰۰	۰/۰۰۰	۰/۰۰۰
B	CWE-122	۳۵	۰/۵۱۵	۰/۲۳۳	۰/۳۲۱
B	CWE-190	۹	۰/۳۳۳	۰/۰۶۰	۰/۱۰۲
C	CWE-122	۳	۰/۶۰۰	۰/۰۲۰	۰/۰۳۹
C	CWE-190	۰	۰/۰۰۰	۰/۰۰۰	۰/۰۰۰
D	CWE-122	۵	۰/۴۵۵	۰/۰۳۳	۰/۰۶۲
D	CWE-190	۰	۰/۰۰۰	۰/۰۰۰	۰/۰۰۰

جدول ۴.۴: نتایج به تفکیک نوع آسیب‌پذیری

مهم‌ترین یافته این بخش آن است که هیچ‌یک از سه پیکربندی مبتنی بر تحلیل ایستا، حتی یک نمونه از ۱۵۰ نمونه سرریز عدد صحیح را کشف نکردند. برای این نوع آسیب‌پذیری، یادآوری هر سه پیکربندی A، C و D دقیقاً صفر است. تنها پیکربندی B توانسته است در این نوع، با یادآوری ۰/۰۶۰ و دقت ۰/۳۳۳، چند نمونه را کشف کند. این تفاوت، ریشه در ماهیت دو نوع آسیب‌پذیری دارد. آسیب‌پذیری سرریز بافر، الگوی شناخته‌شده‌ای دارد و قواعد الگویی ابزارها آن را فعال می‌کنند، در حالی که سرریز عدد صحیح خود یک شکست بنیادی در مدل نوع صحیح را آشکار می‌کند و قاعده‌ای که آن را بگیرد، بسیار دشوارتر است. نتیجه عملی این یافته آن است که ابزارهای ایستا برای پوشش این دسته

از آسیب‌پذیری‌ها به‌تنهایی کافی نیستند و سامانه‌های ترکیبی که در کنار تحلیل ایستا، فایل را به‌طور کامل به مدل زبانی می‌سپارند، پوشش به‌مراتب گسترده‌تری به دست می‌آورند؛ به بهای نرخ بالاتر هشدار روی فایل‌های سالم.

۴.۸ بررسی در مقیاس کامل

برای بررسی وابستگی نتایج فاز یک به اندازه مجموعه، پیکربندی A علاوه بر زیرمجموعه ۶۰۰ نمونه‌ای، روی کل ۴۰۹۸ فایل مجموعه‌داده نیز اجرا شد. نتیجه با مقدار زیرمجموعه هم‌خوان است: دقت ۰/۴۱۲، یادآوری ۰/۰۴۶ و نرخ هشدار روی فایل سالم ۰/۰۵۱. بنابراین محدودیت یادآوری تحلیل ایستا، ویژگی ابزارها و مجموعه‌داده است، نه اثر تصادفی نمونه‌گیری.

۴.۹ ارزیابی موتور شواهد در پایتون

پیکربندی F موتور قطعی را روی ۴۸ فایل پایتون می‌سنجد: ۲۴ سناریوی آسیب‌پذیر و نسخه اصلاح‌شده همان سناریو، در شش دسته دامنه پژوهش. در این آزمایش هیچ فراخوانی مدل زبانی انجام نمی‌شود؛ بنابراین عددها توان شواهد را می‌سنجند، نه توان داوری چندعامله را. نتایج در جدول ۴.۵ آمده است.

پیکربندی	مثبت درست / منفی نادرست	مثبت نادرست / منفی درست	دقت	یادآوری	نرخ هشدار سالم
هر مسیر منبع تا مصرف	24/9	0/15	0.727	1.000	0.375
تنها مسیر بدون کاهش خطر	24/1	0/23	0.960	1.000	0.042
بندیت	16/6	8/18	0.727	0.667	0.250
اجتماع شواهد و بندیت	24/7	0/17	0.774	1.000	0.292

جدول ۴.۵: نتایج موتور قطعی روی محک ۴۸ نمونه‌ای پایتون

اگر هر مسیر بازیابی‌شده شمرده شود، یادآوری کامل است، اما نرخ هشدار روی فایل سالم به ۰/۳۷۵ می‌رسد. با اعمال قانون کاهش خطر، یعنی همان شرطی که دروازه شواهد در گزارش نهایی به کار می‌برد، دقت به ۰/۹۶ و نرخ هشدار به ۰/۰۴۲ می‌رسد و یادآوری همچنان کامل می‌ماند. بندیت در همین مجموعه دقت ۰/۷۲۷، یادآوری ۰/۶۶۷ و نرخ هشدار ۰/۲۵ دارد؛ پس موتور شواهد در این محک کنترل‌شده، هم یادآوری بیشتر و هم هشدار کمتری نسبت به ابزار پایه دارد. این مقایسه معتبر است، اما دامنه آن محدود است: سناریوها مصنوعی‌اند و همراه با موتور نوشته شده‌اند، پس جدول ۴.۵ آزمون بازگشت است نه ارزیابی مستقل.

۴.۱۰ ارزیابی موتور شواهد در جولیت و دیویگن

پیکربندی E همان موتور را روی زیرمجموعه ۶۰۰ نمونه‌ای جولیت می‌سند؛ این زیرمجموعه شامل ۲۹۹ فایل C و ۳۰۱ فایل C++ است. در شمارش مسیرهای بدون کاهش خطر، تفکیک زبانی چنین است: در C دو مثبت درست و سه مثبت نادرست، و در C++ پنج مثبت درست و چهار مثبت نادرست. این تفکیک مهم است، زیرا پشتیبانی نحوی از C++ در غیر این صورت در عدد کلی پنهان می‌ماند. پیکربندی G همان موتور را روی ۲۷۰۲۵۸ تابع واقعی C از مجموعه دیویگن می‌سند. چون این مجموعه برچسب CWE ندارد، تنها می‌توان دقت، یادآوری و نرخ هشدار کلی را گزارش کرد. برای مقایسه رودرو، یک زیرمجموعه متعادل ۶۰۰ تایی نیز با فلافایندر سنجیده شد. در هر سه اجرای پایه، شمار نمونه‌های نامعتبر ابزار صفر بود. نتایج جولیت و دیویگن در جدول ۴.۶ آمده است.

مجموعه و پیکربندی	مثبت درست / مثبت نادرست	منفی نادرست / منفی درست	دقت	یادآوری	نرخ هشدار سالم
جولیت: هر مسیر	26/33	274/267	0.441	0.087	0.110
جولیت: بدون کاهش خطر	7/7	293/293	0.500	0.023	0.023
جولیت: فلافایندر	13/10	287/290	0.565	0.043	0.033
جولیت: اجتماع	13/11	287/289	0.542	0.043	0.037
دیویگن کامل: هر مسیر	971/894	11454/13939	0.521	0.078	0.060
دیویگن کامل: بدون کاهش خطر	861/783	11564/14050	0.524	0.069	0.053
دیویگن متوازن: شواهد	17/21	283/279	0.447	0.057	0.070
دیویگن متوازن: فلافایندر	6/11	294/289	0.353	0.020	0.037
دیویگن متوازن: اجتماع	20/29	280/271	0.408	0.067	0.097

جدول ۴.۶: نتایج موتور قطعی روی جولیت و دیویگن

یادآوری پایین در این دو مجموعه، شکست موتور نیست، بلکه پیامد دامنه آن است. بیشتر موارد سرریز بافر و سرریز عدد صحیح جولیت، ناهماهنگی اندازه و تخصیص‌اند نه جریان داده؛ و برچسب دیویگن به معنای دست‌کاری تابع در یک

وصله امنیتی است، نه لزوماً وجود یک مسیر منبع تا مصرف از جنس دسته‌های مدل شده. به همین دلیل، عدد معنادار این بخش، دقت و نرخ هشدار در کنار پایه ابزار است، نه یادآوری مطلق.

۴.۱۱ تهدیدهای روایی

در تفسیر نتایج باید به چند محدودیت توجه کرد. نخست، اعداد مثبت درست در پیکربندی‌های C و D کوچک‌اند؛ با سه تا پنج مثبت درست، هر تفاوت کوچکی می‌تواند نرخ‌ها را به شدت جابه‌جا کند و این اعداد باید با احتیاط تفسیر شوند. دوم، هر چهار پیکربندی روی یک مدل زبانی مشخص ارزیابی شده‌اند و تعمیم نتایج به سایر مدل‌ها نیازمند آزمایش تکمیلی است. سوم، مجموعه داده جولیت واقعی نیست؛ کد آن قالب‌محور و کوتاه است و همین امر، بازبینی را آسان‌تر از کد واقعی و بزرگ می‌کند. چهارم، چون هر نسخه سالم جولیت با نسخه آسیب‌پذیر هم‌سناریو متناظر است، یک بازبینی که فقط نقطه تفاوت را می‌بیند، ممکن است به‌طور مصنوعی خوب عمل کند؛ به همین دلیل، در پیکربندی C و D تنها قطعه کد پیرامون محل هشدار به عامل داده می‌شود، نه مقایسه با نسخه متناظر. پنجم، محک پایتون مصنوعی و همراه با موتور نوشته شده است، پس نتیجه کامل آن نباید به‌عنوان نرخ واقعی در کد واقعی خوانده شود. ششم، برچسب دیویگن از وصله‌های تاریخی می‌آید و نویز دارد؛ بنابراین دقت و یادآوری آن، تقریبی در برابر همان برچسب است نه حقیقت زمین بازبینی شده. هفتم، پیکربندی‌های چندعامله در این پایان‌نامه به‌صورت مهندسی و با آزمون‌های بدون مدل اعتبارسنجی شده‌اند، اما سنجش تجربی دقت و یادآوری سرتاسری آن‌ها با مدل زنده هنوز انجام نشده است؛ پس اعداد این فصل یا خط پایه تک‌عامله‌اند یا توان موتور قطعی، نه اثبات اثر تفکیک جستجوگر و ارزیاب.

در پایان، نتیجه کلی آزمایش‌ها را می‌توان در یک جمله خلاصه کرد: عامل زبانی، هشدارهای نادرست تحلیل ایستا را به‌طور چشمگیری کاهش می‌دهد، اما این کاهش با افت محسوس یادآوری همراه است، و افزودن شواهد ساختاری این مبادله را به نقطه کاری متعادل‌تری می‌رساند.

فصل پنجم

نتیجه‌گیری و کارهای آینده

۵.۱ جمع‌بندی

در این پژوهش، سامانه‌ای سه‌فاز برای ترکیب تحلیل ساختاری کد با استدلال مدل‌های زبانی بزرگ در قالب معماری چندعامله، به‌منظور کاهش هشدارهای نادرست تحلیل ایستا، طراحی، پیاده‌سازی و ارزیابی شد. خروجی اصلی این سامانه، گزارشی است که تنها یافته‌های دارای شاهد کافی را در بر می‌گیرد و در کنار هر یافته، دلیل و شواهد آن را نیز ارائه می‌کند.

مهم‌ترین یافته تجربی این پژوهش آن است که بازبینی مبتنی بر شواهد، نرخ هشدار روی فایل‌های سالم را از ۰/۰۵۰ به ۰/۰۰۷ رساند؛ یعنی حدود ۸۷ درصد کاهش، و دقت یافته‌ها را از ۰/۳۷۹ به ۰/۶۰۰ افزایش داد. این کاهش اما رایگان نبود: یادآوری از ۰/۳۷ به ۰/۱۰ افت کرد. عامل زبانی، بخشی از یافته‌های درست ابزارهای ایستا را نیز به‌دلیل کمبود شاهد کنار گذاشت.

سه نتیجه دیگر، تصویر کامل‌تری از توان این رویکرد به دست می‌دهند. نخست، تحلیل ایستا به‌تنهایی سقف یادآوری پایینی دارد و این سقف، از توان داوری سامانه مستقل است. دوم، عامل زبانی به‌تنهایی، با یادآوری ۰/۱۴۷، چهار برابر تحلیل ایستا آسیب‌پذیری یافت، اما نرخ هشدار روی فایل سالم آن به ۰/۱۹۷ رسید. تنها پیکربندی‌ای که به هر دو نوع آسیب‌پذیری مجموعه‌داده پوشش داد، پیکربندی B بود؛ به‌ویژه برای سرریز عدد صحیح که هیچ‌یک از پیکربندی‌های مبتنی بر تحلیل ایستا حتی یک نمونه از ۱۵۰ نمونه را کشف نکردند. سوم، موتور قطعی بازتابی مسیر روی محک کنترل‌شده پایتون، با شرط کاهش خطر، دقت ۰/۹۶ و نرخ هشدار ۰/۴۲ را با یادآوری کامل به دست آورد؛ و روی تابع‌های واقعی دیویگن، دقت ۰/۵۲۴ و نرخ هشدار ۰/۰۵۳ را ثبت کرد.

۵.۲ محدودیت‌ها

محدودیت‌های این پژوهش را می‌توان در شش دسته دید. نخست، محدودیت اندازه نمونه؛ تعداد یافته‌های درست در پیکربندی‌های ترکیبی کوچک است و نرخ‌های گزارش‌شده حساس‌اند. دوم، محدودیت مجموعه‌داده؛ کد جولیت قالب‌محور و کوتاه است و یافته‌های آن لزوماً به کد واقعی و بزرگ تعمیم نمی‌یابد؛ محک پایتون مصنوعی است و برچسب دیویگن نیز نويز تاریخی دارد. سوم، محدودیت مدل؛ ارزیابی تنها روی یک مدل زبانی انجام شده است. چهارم، محدودیت سیاست تطبیق؛ با توجه به آنکه برچسب خط در مجموعه‌داده ثبت نشده است، تطبیق بر پایه فایل و خانواده CWE انجام می‌شود و این سیاست، تا حدی سخاوتمندانه است. پنجم، محدودیت درون‌تابعی بودن موتور شواهد؛ پیوند منبع و مصرف در میان تابع‌ها رهگیری نمی‌شود. ششم، معماری چندعامله از نظر مهندسی پیاده‌سازی و آزمون شده است، اما اثر تجربی تفکیک جستجوگر و ارزیاب هنوز با اجرای زنده سنجیده نشده است.

۵.۳ کارهای آینده

بر پایه نتایج این پژوهش، مسیرهای زیر برای ادامه کار پیشنهاد می‌شود.

- بازطراحی بسته شواهد به‌گونه‌ای که به‌جای قطعه‌ای از کد، مسیر داده میان منبع و نقطه خطر را در اختیار عامل قرار دهد. انتظار می‌رود این تغییر، مهم‌ترین گلوگاه فعلی، یعنی کاهش هم‌زمان یادآوری و هشدار نادرست، را برطرف کند.
- استفاده از خروجی نامطمئن به‌جای دور انداختن آن. از آنجا که ۴۲ درصد گروه‌ها در پیکربندی C با حالت نامطمئن بسته شدند، رتبه‌بندی این موارد و ارائه آن‌ها در یک صفحه جداگانه، می‌تواند بخشی از یادآوری از دست‌رفته را بدون افزودن هشدار نادرست بازیابی کند.
- تنظیم نقطه کاری سامانه بر پایه هزینه نسبی خطا. با آستانه متفاوت برای تأیید، می‌توان سامانه را برای محیط‌هایی که هشدار نادرست پرهزینه‌تر است، یا محیط‌هایی که از دست رفتن آسیب‌پذیری پرهزینه‌تر است، تنظیم کرد.
- ارزیابی روی مجموعه داده‌های دیگر و کد واقعی. تکرار آزمایش‌ها روی مجموعه داده‌هایی با کد بلندتر و وابستگی بین‌فایلی، مشخص می‌کند که نتایج تا چه اندازه به ویژگی جولیت وابسته‌اند.
- توسعه بازیابی تک‌مرحله‌ای به یک عامل چندمرحله‌ای با ابزار پرس‌وجو. اگر عامل بتواند به‌جای اتکا به قطعه ثابت کد، خود زمینه لازم را از فایل و ساختار پروژه استخراج کند، انتظار می‌رود نرخ خروجی نامطمئن کاهش یابد.
- اجرای زنده معماری جستجوگر و ارزیاب روی همان محک‌های پایتون و C و C++ و مقایسه آن با مسیر تک‌عامله، تا سهم تفکیک نقش‌ها از سهم شواهد جدا شود.

فهرست مراجع

- [١] cppcheck, "A tool for static C/C++ code analysis." [Online]. Available: <https://cppcheck.sourceforge.io/>
- [٢] flawfinder, "A free, open source C/C++ program analyzer." [Online]. Available: <https://sourceforge.net/projects/flawfinder/>
- [٣] The clang static analyzer, "A static analyzer for C/C++ built on top of the LLVM compiler infrastructure." [Online]. Available: <https://clang-analyzer.llvm.org/>
- [٤] National Institute of Standards and Technology, "Juliet C/C++ Test Suite," in Software Assurance and Vulnerability Metrics, SARD test suites. [Online]. Available: <https://samate.nist.gov/SARD/test-suites/112>
- [٥] MITRE, "CWE-122: Heap-based Buffer Overflow." [Online]. Available: <https://cwe.mitre.org/data/definitions/122.html>
- [٦] B. Johnson, Y. Song, E. Murphy-Hill, and R. Bowdidge, "Why don't software developers use static analysis tools to find bugs?," in Proc. 35th Int. Conf. Software Engineering (ICSE), 2013, pp. 672-681.
- [٧] N. Ayewah and W. Pugh, "The Google FindBugs fixit," in Proc. 19th Int. Symp. Software Testing and Analysis (ISSTA), 2010.
- [٨] D. Evans and D. Larochelle, "Improving security using extensible lightweight static analysis," in IEEE Software, vol. 19, no. 1, 2002, pp. 42-51.
- [٩] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," in Advances in Neural Information Processing Systems 32 (NeurIPS), 2019, pp. 10197-10207.
- [١٠] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions," in Proc. IEEE Symp. Security and Privacy (SP), 2022, pp. 754-768.

- [۱۱] A. Fan, B. Gokkaya, M. Harman, M. Lyubarskiy, S. Sengupta, S. Yoo, and J. Zhang, “Large language models for software engineering: Survey and open problems,” in Proc. IEEE/ACM Int. Conf. Software Engineering: Future of Software Engineering (ICSE-FoSE), 2023, pp. 31-53.
- [۱۲] Tree-sitter contributors, “Tree-sitter documentation: an incremental parsing system for programming tools.” [Online]. Available: <https://tree-sitter.github.io/tree-sitter/>
- [۱۳] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 9459-9474.
- [۱۴] E. M. Bender et al., “On the dangers of stochastic parrots: Can language models be too big?,” in Proc. ACM Fairness, Accountability, and Transparency, 2021, pp. 610-623.
- [۱۵] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, “Do users write more insecure code with AI assistants?,” in Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS), 2023, pp. 2785-2799.
- [۱۶] National Institute of Standards and Technology, “CVE-2021-3156 Detail,” National Vulnerability Database. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2021-3156>
- [۱۷] National Institute of Standards and Technology, “CVE-2017-7529 Detail,” National Vulnerability Database. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2017-7529>
- [۱۸] National Institute of Standards and Technology, “CVE-2022-22817 Detail,” National Vulnerability Database. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2022-22817>
- [۱۹] National Institute of Standards and Technology, “CVE-2025-27516 Detail,” National Vulnerability Database. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2025-27516>
- [۲۰] National Institute of Standards and Technology, “CVE-2017-18342 Detail,” National Vulnerability Database. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2017-18342>
- [۲۱] National Institute of Standards and Technology, “CVE-2021-25239 Detail,” National Vulnerability Database. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2021-25239>